

Physlib Monthly Report

1–30 September 2024

Contributors: Joseph Tooby-Smith, Pietro Monticone

Generated 14 August 2026 from commit 94ceb30.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–30 September 2024 on branch `master` of [leanprover-community/physlib](#), between commits `86b3b8a` and `94ceb30`. Over that period 2 contributors landed 124 commits, changing 5,246 lines across 106 files and adding 202 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>Archive.Papers.1907_00514</code>	3
2.2	<code>HepLean.BeyondTheStandardModel.GeorgiGlashow.Basic</code>	5
2.3	<code>HepLean.BeyondTheStandardModel.PatiSalam.Basic</code>	6
2.4	<code>HepLean.BeyondTheStandardModel.Spin10.Basic</code>	7
2.5	<code>HepLean.BeyondTheStandardModel.TwoHDM.Basic</code>	8
2.6	<code>HepLean.BeyondTheStandardModel.TwoHDM.GaugeOrbits</code>	10
2.7	<code>HepLean.Meta.AllFilePaths</code>	11
2.8	<code>HepLean.Meta.Informal</code>	11
2.9	<code>HepLean.Meta.TransverseTactics</code>	12
2.10	<code>HepLean.SpaceTime.LorentzGroup.Proper</code>	13
2.11	<code>HepLean.SpaceTime.LorentzGroup.Restricted</code>	13
2.12	<code>HepLean.SpaceTime.SL2C.Basic</code>	13
2.13	<code>HepLean.SpaceTime.WeylFermion.Basic</code>	14
2.14	<code>HepLean.StandardModel.Basic</code>	16
2.15	<code>HepLean.StandardModel.HiggsBoson.Basic</code>	18
2.16	<code>HepLean.StandardModel.HiggsBoson.GaugeAction</code>	19
2.17	<code>HepLean.StandardModel.HiggsBoson.PointwiseInnerProd</code>	20
2.18	<code>HepLean.StandardModel.HiggsBoson.Potential</code>	21
2.19	<code>scripts.MetaPrograms.check_rfl</code>	25
2.20	<code>scripts.MetaPrograms.free_simps</code>	25
2.21	<code>scripts.MetaPrograms.informal</code>	26

1 Introduction

During Sep 2024, 2 contributors submitted 124 commits that changed 5,246 lines (+4,181 / -1,065) across 106 files spanning 20 top-level areas of the repository. Of those, 13 files were newly added. This report catalogues 202 newly added Lean declarations: 67 defs, 54 lemmas, 36 informal definitions, 27 informal lemmas, 11 structures, 3 theorems, 2 instances, 1 abbrev, and 1 inductive.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	6,955	114
Pietro Monticone	155	10

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

No reviewers identified for this month.

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
Archive/Papers	1	+1	–	+913	-0
HepLean/Tensors	24	–	–	+508	-298
HepLean/StandardModel	5	–	–	+565	-182
scripts/MetaPrograms	3	+3	–	+688	-0
HepLean/AnomalyCancellation	29	–	–	+274	-320
HepLean/BeyondTheStandardModel	5	+4	–	+375	-58
HepLean/SpaceTime	18	+1	–	+332	-84
HepLean/Meta	3	+3	–	+310	-0
HepLean/FlavorPhysics	6	–	–	+99	-62
HepLean.lean	1	–	–	+32	-24
.github/workflows	1	–	–	+23	-15
docs	1	+1	–	+24	-0
HepLean/Mathematics	2	–	–	+9	-8
lake-manifest.json	1	–	–	+6	-6
lakefile.toml	1	–	–	+10	-1
docs/assets	1	–	–	+7	-3
README.md	1	–	–	+2	-1
docs/_layouts	1	–	–	+2	-1
HepLean/FeynmanDiagrams	1	–	–	+1	-1
lean-toolchain	1	–	–	+1	-1

1.4 New declarations by kind

Kind	Count
def	67
lemma	54
informal_definition	36
informal_lemma	27
structure	11
theorem	3
instance	2
abbrev	1
inductive	1

2 Details by file

2.1 Archive.Papers.1907_00514

[Archive/Papers/1907_00514.lean](#) on GitHub.

structure ACCSystemCharges

[\[source\]](#)

A system of charges, specified by the number of charges.

```
structure ACCSystemCharges where
  numberCharges : ℕ
```

def ACCSystemChargesMk

[\[source\]](#)

Creates an ACCSystemCharges object with the specified number of charges.

```
def ACCSystemChargesMk (n : ℕ) : ACCSystemCharges where
```

def Charges

[\[source\]](#)

The charges as functions from Fin χ .numberCharges $\rightarrow \mathbb{Q}$.

```
def Charges ( $\chi$  : ACCSystemCharges) : Type
```

instance chargesAddCommMonoid

[\[source\]](#)

An instance to provide the necessary operations and properties for charges to form an additive commutative monoid.

```
@[simps!]
instance chargesAddCommMonoid ( $\chi$  : ACCSystemCharges) : AddCommMonoid  $\chi$ .Charges
```

instance chargesModule

[\[source\]](#)

An instance to provide the necessary operations and properties for charges to form a module over the field $\mathbb{Q}$.

```
@[simps!]
instance chargesModule ( $\chi$  : ACCSystemCharges) : Module  $\mathbb{Q}$   $\chi$ .Charges
```

structure ACCSystemLinear[\[source\]](#)*The type of charges plus the linear ACCs.*

```

structure ACCSystemLinear extends ACCSystemCharges where
  numberLinear : ℕ
  linearACCs : Fin numberLinear → (toACCSystemCharges.Charges →l [Q] Q)

```

structure LinSols[\[source\]](#)*The type of solutions to the linear ACCs.*

```

structure LinSols (χ : ACCSystemLinear) where
  val : χ.l.Charges
  linearSol : ∀ i : Fin χ.numberLinear, χ.linearACCs i val = 0

```

lemma ext[\[source\]](#)*Two solutions are equal if the underlying charges are equal.*

```

@[ext]
lemma LinSols.ext {χ : ACCSystemLinear} {S T : χ.LinSols} (h : S.val = T.val) : S = T

```

structure ACCSystemQuad[\[source\]](#)*The type of charges plus the linear ACCs plus the quadratic ACCs.*

```

structure ACCSystemQuad extends ACCSystemLinear where
  numberQuadratic : ℕ
  quadraticACCs : Fin numberQuadratic → HomogeneousQuadratic toACCSystemCharges.Charges

```

structure QuadSols[\[source\]](#)*The type of solutions to the linear and quadratic ACCs.*

```

structure QuadSols (χ : ACCSystemQuad) extends χ.LinSols where
  quadSol : ∀ i : Fin χ.numberQuadratic, (χ.quadraticACCs i) val = 0

```

structure ACCSystem[\[source\]](#)*The type of charges plus the anomaly cancellation conditions.*

```

structure ACCSystem extends ACCSystemQuad where
  cubicACC : HomogeneousCubic toACCSystemCharges.Charges

```

structure Sols[\[source\]](#)*The type of solutions to the anomaly cancellation conditions.*

```

structure Sols (χ : ACCSystem) extends χ.QuadSols where
  cubicSol : χ.cubicACC val = 0

```

lemma E_zero_iff_Q_zero[\[source\]](#)

```
lemma E_zero_iff_Q_zero {S : (SMNoGrav 1).Sols} : Q S.val (0 : Fin 1) = 0 ↔
  E S.val (0 : Fin 1) = 0
```

lemma accGrav_Q_zero[\[source\]](#)

```
lemma accGrav_Q_zero {S : (SMNoGrav 1).Sols} (hQ : Q S.val (0 : Fin 1) = 0) :
  accGrav S.val = 0
```

lemma accGrav_Q_neq_zero[\[source\]](#)

```
lemma accGrav_Q_neq_zero {S : (SMNoGrav 1).Sols} (hQ : Q S.val (0 : Fin 1) ≠ 0)
  (FLTThree : FermatLastTheoremWith Q 3) :
  accGrav S.val = 0
```

theorem accGravSatisfied[\[source\]](#)

Any solution to the ACCs without gravity satisfies the gravitational ACC.

```
theorem accGravSatisfied {S : (SMNoGrav 1).Sols} (FLTThree : FermatLastTheoremWith Q 3) :
  accGrav S.val = 0
```

2.2 HepLean.BeyondTheStandardModel.GeorgiGlashow.Basic

[HepLean/BeyondTheStandardModel/GeorgiGlashow/Basic.lean](#) on GitHub.

informal_definition GaugeGroupI[\[source\]](#)

```
informal_definition GaugeGroupI where
  math := "The group `SU(5)`."
  physics := "The gauge group of the Georgi-Glashow model."
```

informal_definition inclSM[\[source\]](#)

```
informal_definition inclSM where
  physics := "The homomorphism of the Standard Model gauge group into the
    Georgi-Glashow gauge group."
  math := "The group homomorphism `SU(3) x SU(2) x U(1) -> SU(5)`
    taking (h, g, α) to (blockdiag (α ^ 3 g, α ^ (-2) h))."
  ref := "Page 34 of https://math.ucr.edu/home/baez/guts.pdf"
  deps := [GaugeGroupI, StandardModel.GaugeGroupI]
```

informal_lemma inclSM_ker[\[source\]](#)

```
informal_lemma inclSM_ker where
  math := "The kernel of the map ``inclSM is equal to the subgroup
    ``StandardModel.gaugeGroupZ6SubGroup."
  ref := "Page 34 of https://math.ucr.edu/home/baez/guts.pdf"
  deps := [inclSM, StandardModel.gaugeGroupZ6SubGroup]
```

informal_definition embedSM $\mathbb{Z}_6$ [\[source\]](#)

```
informal_definition embedSM $\mathbb{Z}_6$  where
  math := "The group embedding from ``StandardModel.GaugeGroup $\mathbb{Z}_6$  to ``GaugeGroupI
    induced by ``inclSM by quotienting by the kernel ``inclSM_ker."
  deps := [``inclSM, ``StandardModel.GaugeGroup $\mathbb{Z}_6$ , ``GaugeGroupI, ``inclSM_ker]
```

2.3 HepLean.BeyondTheStandardModel.PatiSalam.Basic

[HepLean/BeyondTheStandardModel/PatiSalam/Basic.lean](#) on GitHub.

informal_definition GaugeGroupI[\[source\]](#)

```
informal_definition GaugeGroupI where
  math := "The group `SU(4) x SU(2) x SU(2)`."
  physics := "The gauge group of the Pati-Salam model (unquotiented by  $\mathbb{Z}_2$ )."
```

informal_definition inclSM[\[source\]](#)

```
informal_definition inclSM where
  physics := "The homomorphism of the Standard Model gauge group into the Pati-Salam gauge
    group."
  math := "The group homomorphism `SU(3) x SU(2) x U(1) -> SU(4) x SU(2) x SU(2)`
    taking (h, g,  $\alpha$ ) to (blockdiag ( $\alpha$  h,  $\alpha$  ^ (-3)), g, diag( $\alpha$  ^ (3),  $\alpha$  ^ (-3)))."
  ref := "Page 54 of https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI, ``StandardModel.GaugeGroupI]
```

informal_lemma inclSM_ker[\[source\]](#)

```
informal_lemma inclSM_ker where
  math := "The kernel of the map ``inclSM is equal to the subgroup
    ``StandardModel.gaugeGroup $\mathbb{Z}_3$ SubGroup."
  ref := "Footnote 10 of https://arxiv.org/pdf/2201.07245"
  deps := [``inclSM, ``StandardModel.gaugeGroup $\mathbb{Z}_3$ SubGroup]
```

informal_definition embedSM $\mathbb{Z}_3$ [\[source\]](#)

```
informal_definition embedSM $\mathbb{Z}_3$  where
  math := "The group embedding from ``StandardModel.GaugeGroup $\mathbb{Z}_3$  to ``GaugeGroupI
    induced by ``inclSM by quotienting by the kernel ``inclSM_ker."
  deps := [``inclSM, ``StandardModel.GaugeGroup $\mathbb{Z}_3$ , ``GaugeGroupI, ``inclSM_ker]
```

informal_definition gaugeGroupISpinEquiv[\[source\]](#)

```
informal_definition gaugeGroupISpinEquiv where
  math := "The equivalence between `GaugeGroupI` and `Spin(6) x Spin(4)`."
  deps := [``GaugeGroupI]
```

informal_definition gaugeGroup $\mathbb{Z}_2$ SubGroup[\[source\]](#)

```
informal_definition gaugeGroup $\mathbb{Z}_2$ SubGroup where
  physics := "The  $\mathbb{Z}_2$ -subgroup of the un-quotiented gauge group which acts trivially on
    all particles in the standard model."
  math := "The  $\mathbb{Z}_2$ -subgroup of ``GaugeGroupI with the non-trivial element (-1, -1, -1)."
```

```
ref := "https://math.ucr.edu/home/baez/guts.pdf"
deps := [``GaugeGroupI]
```

informal_definition GaugeGroupZ₂[\[source\]](#)**informal_definition GaugeGroupZ₂ where**

```
physics := "The gauge group of the Pati-Salam model with a Z2 quotient."
math := "The quotient of ``GaugeGroupI by the Z2-subgroup `gaugeGroupZ2SubGroup`."
ref := "https://math.ucr.edu/home/baez/guts.pdf"
deps := [``GaugeGroupI, ``gaugeGroupZ2SubGroup]
```

informal_lemma sm_Z₆_factor_through_gaugeGroupZ₂SubGroup[\[source\]](#)**informal_lemma sm_Z₆_factor_through_gaugeGroupZ₂SubGroup where**

```
math := "The group ``StandardModel.gaugeGroupZ6SubGroup under the homomorphism ``embedSM
  factors
  through the subgroup ``gaugeGroupZ2SubGroup."
deps := [``inclSM, ``StandardModel.gaugeGroupZ6SubGroup, ``gaugeGroupZ2SubGroup]
```

informal_definition embedSMZ₆ToZ₂[\[source\]](#)**informal_definition embedSMZ₆ToZ₂ where**

```
math := "The group homomorphism from ``StandardModel.GaugeGroupZ6 to ``GaugeGroupZ2
  induced by ``embedSM."
deps := [``inclSM, ``StandardModel.GaugeGroupZ6, ``GaugeGroupZ2,
  ``sm_Z6_factor_through_gaugeGroupZ2SubGroup]
```

2.4 HepLean.BeyondTheStandardModel.Spin10.Basic

[HepLean/BeyondTheStandardModel/Spin10/Basic.lean](#) on GitHub.

informal_definition GaugeGroupI[\[source\]](#)**informal_definition GaugeGroupI where**

```
math := "The group `Spin(10)`."
physics := "The gauge group of the Spin(10) model (aka S0(10)-model.)"
```

informal_definition inclPatiSalam[\[source\]](#)**informal_definition inclPatiSalam where**

```
physics := "The inclusion of the Pati-Salam gauge group into Spin(10)."
```

$$\text{math} := \text{"The lift of the embedding } S0(6) \times S0(4) \rightarrow S0(10) \text{ to universal covers,} \\ \text{giving a homomorphism } Spin(6) \times Spin(4) \rightarrow Spin(10) \text{ . Precomposed with the isomorphism,} \\ \text{``PatiSalam.gaugeGroupISpinEquiv, between } SU(4) \times SU(2) \times SU(2) \text{ and } Spin(6) \times Spin(4) \text{ .}"}$$

```
ref := "Page 56 of https://math.ucr.edu/home/baez/guts.pdf"
deps := [``GaugeGroupI, ``PatiSalam.GaugeGroupI, ``PatiSalam.gaugeGroupISpinEquiv]
```

informal_definition inclSM[\[source\]](#)**informal_definition inclSM where**

```
physics := "The inclusion of the Standard Model gauge group into Spin(10)."
```

$$\text{math} := \text{"The compoisiton of ``embedPatiSalam and ``PatiSalam.inclSM."}$$

```
ref := "Page 56 of https://math.ucr.edu/home/baez/guts.pdf"
```

```
deps := [``inclPatiSalam, ``PatiSalam.inclSM]
```

informal_definition inclGeorgiGlashow

[source]

informal_definition inclGeorgiGlashow **where**

```
physics := "The inclusion of the Georgi-Glashow gauge group into Spin(10)."  
math := "The Lie group homomorphism from SU(n) → Spin(2n) dicussed on page 46 of  
https://math.ucr.edu/home/baez/guts.pdf for n = 5."  
deps := [``GaugeGroupI, ``GeorgiGlashow.GaugeGroupI]
```

informal_definition inclSMThruGeorgiGlashow

[source]

informal_definition inclSMThruGeorgiGlashow **where**

```
physics := "The inclusion of the Standard Model gauge group into Spin(10)."  
math := "The composition of ``inclGeorgiGlashow and ``GeorgiGlashow.inclSM."  
deps := [``inclGeorgiGlashow, ``GeorgiGlashow.inclSM]
```

informal_lemma inclSM_eq_inclSMThruGeorgiGlashow

[source]

informal_lemma inclSM_eq_inclSMThruGeorgiGlashow **where**

```
math := "The inclusion ``inclSM is equal to the inclusion ``inclSMThruGeorgiGlashow."  
deps := [``inclSM, ``inclSMThruGeorgiGlashow]
```

2.5 HepLean.BeyondTheStandardModel.TwoHDM.Basic

[HepLean/BeyondTheStandardModel/TwoHDM/Basic.lean](#) on GitHub.

structure Potential

[source]

The parameters of the Two Higgs doublet model potential.

structure Potential **where**

```
m112 : ℝ  
m222 : ℝ  
m122 : ℂ  
↑↓1 : ℝ  
↑↓2 : ℝ  
↑↓3 : ℝ  
↑↓4 : ℝ  
↑↓5 : ℂ  
↑↓6 : ℂ  
↑↓7 : ℂ
```

def toFun

[source]

The potential of the two Higgs doublet model.

```
def toFun (ϕ1 ϕ2 : HiggsField) (x : SpaceTime) : ℝ
```

def zero

[source]

The potential where all parameters are zero.

```
def zero : Potential
```


lemma swap_fields[\[source\]](#)

```
lemma swap_fields : P.toFun  $\Phi_1$   $\Phi_2$  =
  (Potential.mk P.m222 P.m112 (conj P.m122) P. $\uparrow_2$  P. $\uparrow_1$  P. $\uparrow_3$  P. $\uparrow_4$ 
   (conj P. $\uparrow_5$ ) (conj P. $\uparrow_7$ ) (conj P. $\uparrow_6$ )).toFun  $\Phi_2$   $\Phi_1$ 
```

lemma right_zero[\[source\]](#)

If Φ_2 is zero the potential reduces to the Higgs potential on Φ_1 .

```
lemma right_zero : P.toFun  $\Phi_1$  0 =
  (HiggsField.Potential.mk (- P.m112) (P. $\uparrow_1$ /2)).toFun  $\Phi_1$ 
```

lemma left_zero[\[source\]](#)

If Φ_1 is zero the potential reduces to the Higgs potential on Φ_2 .

```
lemma left_zero : P.toFun 0  $\Phi_2$  =
  (HiggsField.Potential.mk (- P.m222) (P. $\uparrow_2$ /2)).toFun  $\Phi_2$ 
```

lemma neg_left[\[source\]](#)

Negating Φ_1 is equivalent to negating m_{122} , $\uparrow_6$ and $\uparrow_7$.

```
lemma neg_left : P.toFun (-  $\Phi_1$ )  $\Phi_2$ 
= (Potential.mk P.m112 P.m222 (- P.m122) P. $\uparrow_1$  P. $\uparrow_2$  P. $\uparrow_3$  P. $\uparrow_4$  P. $\uparrow_5$  (- P. $\uparrow_6$ ) (- P. $\uparrow_7$ )).toFun
 $\Phi_1$   $\Phi_2$ 
```

lemma neg_right[\[source\]](#)

Negating Φ_2 is equivalent to negating m_{122} , $\uparrow_6$ and $\uparrow_7$.

```
lemma neg_right : P.toFun  $\Phi_1$  (-  $\Phi_2$ )
= (Potential.mk P.m112 P.m222 (- P.m122) P. $\uparrow_1$  P. $\uparrow_2$  P. $\uparrow_3$  P. $\uparrow_4$  P. $\uparrow_5$  (- P. $\uparrow_6$ ) (- P. $\uparrow_7$ )).toFun
 $\Phi_1$   $\Phi_2$ 
```

lemma left_eq_right[\[source\]](#)

```
lemma left_eq_right : P.toFun  $\Phi_1$   $\Phi_1$  =
  (HiggsField.Potential.mk (- P.m112 - P.m222 + 2 * P.m122.re)
   (P. $\uparrow_1$ /2 + P. $\uparrow_2$ /2 + P. $\uparrow_3$  + P. $\uparrow_4$  + P. $\uparrow_5$ .re + 2 * P. $\uparrow_6$ .re + 2 * P. $\uparrow_7$ .re)).toFun  $\Phi_1$ 
```

lemma left_eq_neg_right[\[source\]](#)

```
lemma left_eq_neg_right : P.toFun  $\Phi_1$  (-  $\Phi_1$ ) =
  (HiggsField.Potential.mk (- P.m112 - P.m222 - 2 * P.m122.re)
   (P. $\uparrow_1$ /2 + P. $\uparrow_2$ /2 + P. $\uparrow_3$  + P. $\uparrow_4$  + P. $\uparrow_5$ .re - 2 * P. $\uparrow_6$ .re - 2 * P. $\uparrow_7$ .re)).toFun  $\Phi_1$ 
```

def IsBounded[\[source\]](#)

The proposition on the coefficients for a potential to be bounded.

```
def IsBounded : Prop
```

lemma isBounded_right_zero[\[source\]](#)

```
lemma isBounded_right_zero (h : P.IsBounded) :
  (HiggsField.Potential.mk (- P.m112) (P.↑1/2)).IsBounded
```

lemma isBounded_left_zero[\[source\]](#)

```
lemma isBounded_left_zero (h : P.IsBounded) :
  (HiggsField.Potential.mk (- P.m222) (P.↑2/2)).IsBounded
```

lemma isBounded_↑₁_nonneg[\[source\]](#)

```
lemma isBounded_↑1_nonneg (h : P.IsBounded) :
  0 ≤ P.↑1
```

lemma isBounded_↑₂_nonneg[\[source\]](#)

```
lemma isBounded_↑2_nonneg (h : P.IsBounded) :
  0 ≤ P.↑2
```

lemma isBounded_of_left_eq_right[\[source\]](#)

```
lemma isBounded_of_left_eq_right (h : P.IsBounded) :
  0 ≤ P.↑1/2 + P.↑2/2 + P.↑3 + P.↑4 + P.↑5.re + 2 * P.↑6.re + 2 * P.↑7.re
```

lemma isBounded_of_left_eq_neg_right[\[source\]](#)

```
lemma isBounded_of_left_eq_neg_right (h : P.IsBounded) :
  0 ≤ P.↑1/2 + P.↑2/2 + P.↑3 + P.↑4 + P.↑5.re - 2 * P.↑6.re - 2 * P.↑7.re
```

lemma nonneg_sum_↑₁_to_↑₅_of_isBounded[\[source\]](#)

```
lemma nonneg_sum_↑1_to_↑5_of_isBounded (h : P.IsBounded) :
  0 ≤ P.↑1/2 + P.↑2/2 + P.↑3 + P.↑4 + P.↑5.re
```

2.6 HepLean.BeyondTheStandardModel.TwoHDM.GaugeOrbits

[HepLean/BeyondTheStandardModel/TwoHDM/GaugeOrbits.lean](#) on GitHub.

informal_definition prodMatrix[\[source\]](#)

```
informal_definition prodMatrix where
  math := "For two Higgs fields `Φ1` and `Φ2`, the map from space time to 2 x 2 complex matrices"
  defined by ((Φ1^†Φ1, Φ2^†Φ1), (Φ1^†Φ2, Φ2^†Φ2)). "
  ref := "https://arxiv.org/pdf/hep-ph/0605184 eq 3.8."
  deps := [``StandardModel.HiggsVec, ``SpaceTime]
```

informal_lemma prodMatrix_positive_semidefinite[\[source\]](#)

```
informal_lemma prodMatrix_positive_semidefinite where
  math := "For all x in ``SpaceTime, ``prodMatrix at `x` is positive semidefinite."
```

```
deps := [``prodMatrix, ``SpaceTime]
```

informal_lemma prodMatrix_hermitian

[\[source\]](#)

```
informal_lemma prodMatrix_hermitian where
  math := "For all x in ``SpaceTime, ``prodMatrix at `x` is hermitian."
  deps := [``prodMatrix, ``SpaceTime]
```

informal_lemma prodMatrix_smooth

[\[source\]](#)

```
informal_lemma prodMatrix_smooth where
  math := "The map ``prodMatrix is a smooth function on spacetime."
  deps := [``prodMatrix]
```

2.7 HepLean.Meta.AllFilePaths

[HepLean/Meta/AllFilePaths.lean](#) on GitHub.

def go

[\[source\]](#)

The recursive function underlying allFilePaths.

```
partial def allFilePaths.go (prev : Array FilePath)
  (root : Name) (path : FilePath) : IO (Array FilePath)
```

def allFilePaths

[\[source\]](#)

Gets an array of all file paths in HepLean.

```
partial def allFilePaths : IO (Array FilePath)
```

2.8 HepLean.Meta.Informal

[HepLean/Meta/Informal.lean](#) on GitHub.

structure InformalDefinition

[\[source\]](#)

The structure representating an informal definition.

```
structure InformalDefinition where
  name : Name
  math : String
  physics : String
  ref : String
  dependencies : List Name
```

structure InformalLemma

[\[source\]](#)

The structure representating an informal proof.

```
structure InformalLemma where
  name : Name
  math : String
  physics : String
```

```
proof : String
ref : String
dependencies : List Name
```

def \$name [\[source\]](#)

An informal lemma.

```
def $name : InformalLemma
```

def isInformal [\[source\]](#)

Is true if and only if a ConstantInfo corresponds to an InformalLemma or a InformalDefinition.

```
def isInformal (c : ConstantInfo) : Bool
```

def isInformalLemma [\[source\]](#)

Is true if and only if a ConstantInfo corresponds to an InformalLemma.

```
def isInformalLemma (c : ConstantInfo) : Bool
```

def isInformalDef [\[source\]](#)

Is true if and only if a ConstantInfo corresponds to an InformalDefinition.

```
def isInformalDef (c : ConstantInfo) : Bool
```

2.9 HepLean.Meta.TransverseTactics

[HepLean/Meta/TransverseTactics.lean](#) on GitHub.

def processCommands [\[source\]](#)

Takes in a file and returns the infostates of commands and the corresponding environment before the command is processed.

```
partial def processCommands : Frontend.FrontendM (List (Environment × InfoState))
```

def visitInfo [\[source\]](#)

Tests if a given info is ofTacticInfo and if so runs visitTacticInfo.

```
def visitInfo (file : FilePath) (env : Environment)
  (visitTacticInfo : FilePath → ContextInfo → TacticInfo → MetaM Unit) (ci : ContextInfo)
  (info : Info) (acc : List (IO Unit)) : List (IO Unit)
```

def traverseForest [\[source\]](#)

Applies visitInfo to each node of the info trees.

```
def traverseForest (file : FilePath)
  (visitTacticInfo : FilePath → ContextInfo → TacticInfo → MetaM Unit)
  (steps : List (Environment × InfoState)) : List (IO Unit)
```

def transverseTactics[\[source\]](#)*Applies visitTacticInfo to each tactic in a file.*

```

unsafe def transverseTactics (file : FilePath)
  (visitTacticInfo : FilePath → ContextInfo → TacticInfo → MetaM Unit) : IO Unit

```

2.10 HepLean.SpaceTime.LorentzGroup.Proper[HepLean/SpaceTime/LorentzGroup/Proper.lean](#) on GitHub.**lemma detContinuous_eq_one**[\[source\]](#)

```

lemma detContinuous_eq_one (Λ : LorentzGroup d) :
  detContinuous Λ = Additive.toMul 0 ↔ Λ.1.det = 1

```

lemma detContinuous_eq_zero[\[source\]](#)

```

lemma detContinuous_eq_zero (Λ : LorentzGroup d) :
  detContinuous Λ = Additive.toMul (1 : ZMod 2) ↔ Λ.1.det = - 1

```

2.11 HepLean.SpaceTime.LorentzGroup.Restricted[HepLean/SpaceTime/LorentzGroup/Restricted.lean](#) on GitHub.**informal_definition Restricted**[\[source\]](#)

```

informal_definition Restricted where
  math := "The subgroup of the Lorentz group consisting of elements which
    are proper and orthochronous."
  deps := [``LorentzGroup, ``IsProper, ``IsOrthochronous]

```

2.12 HepLean.SpaceTime.SL2C.Basic[HepLean/SpaceTime/SL2C/Basic.lean](#) on GitHub.**informal_lemma toLorentzGroup_det_one**[\[source\]](#)

```

informal_lemma toLorentzGroup_det_one where
  math := "The determinant of the image of `SL(2, C)` in the Lorentz group is one."
  deps := [``toLorentzGroup]

```

informal_lemma toLorentzGroup_timeComp_nonneg[\[source\]](#)

```

informal_lemma toLorentzGroup_timeComp_nonneg where
  math := "The time coponent of the image of `SL(2, C)` in the Lorentz group is non-negative."
  deps := [``toLorentzGroup, ``LorentzGroup.timeComp]

```

informal_lemma toRestrictedLorentzGroup[\[source\]](#)

```

informal_lemma toRestrictedLorentzGroup where
  math := "The homomorphism from `SL(2, C)` to the restricted Lorentz group."
  deps := [``toLorentzGroup, ``toLorentzGroup_det_one, ``toLorentzGroup_timeComp_nonneg,
    ``LorentzGroup.Restricted]

```

2.13 HepLean.SpaceTime.WeylFermion.Basic

[HepLean/SpaceTime/WeylFermion/Basic.lean](#) on GitHub.

informal_definition leftHandedWeyl

[\[source\]](#)

```
informal_definition leftHandedWeyl where
  math := "The vector space  $\mathbb{C}^2$  carrying the fundamental representation of  $SL(2, \mathbb{C})$ ."
  physics := "A Weyl fermion with indices  $\psi_a$ ."
  ref := "https://particle.physics.ucdavis.edu/modernsusy/slides/slideimages/spinorfeynrules.pdf"
```

informal_definition rightHandedWeyl

[\[source\]](#)

```
informal_definition rightHandedWeyl where
  math := "The vector space  $\mathbb{C}^2$  carrying the conjugate representation of  $SL(2, \mathbb{C})$ ."
  physics := "A Weyl fermion with indices  $\psi_{\dot{a}}$ ."
  ref := "https://particle.physics.ucdavis.edu/modernsusy/slides/slideimages/spinorfeynrules.pdf"
```

informal_definition altLeftHandedWeyl

[\[source\]](#)

```
informal_definition altLeftHandedWeyl where
  math := "The vector space  $\mathbb{C}^2$  carrying the representation of  $SL(2, \mathbb{C})$  given by  

 $M \rightarrow (M^{-1})^T$ ."
  physics := "A Weyl fermion with indices  $\psi^a$ ."
  ref := "https://particle.physics.ucdavis.edu/modernsusy/slides/slideimages/spinorfeynrules.pdf"
```

informal_definition altRightHandedWeyl

[\[source\]](#)

```
informal_definition altRightHandedWeyl where
  math := "The vector space  $\mathbb{C}^2$  carrying the representation of  $SL(2, \mathbb{C})$  given by  

 $M \rightarrow (M^{-1})^\dagger$ ."
  physics := "A Weyl fermion with indices  $\psi^{\dot{a}}$ ."
  ref := "https://particle.physics.ucdavis.edu/modernsusy/slides/slideimages/spinorfeynrules.pdf"
```

informal_definition leftHandedWeylAltEquiv

[\[source\]](#)

```
informal_definition leftHandedWeylAltEquiv where
  math := "The linear equiv between leftHandedWeyl and altLeftHandedWeyl given  

  by multiplying an element of rightHandedWeyl by the matrix  $\epsilon^{a\dot{b}} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ ."
  deps := [leftHandedWeyl, altLeftHandedWeyl]
```

informal_lemma leftHandedWeylAltEquiv_equivariant

[\[source\]](#)

```
informal_lemma leftHandedWeylAltEquiv_equivariant where
  math := "The linear equiv leftHandedWeylAltEquiv is equivariant with respect to the  

  action of  $SL(2, \mathbb{C})$  on leftHandedWeyl and altLeftHandedWeyl."
  deps := [leftHandedWeylAltEquiv]
```

informal_definition rightHandedWeylAltEquiv[\[source\]](#)

```
informal_definition rightHandedWeylAltEquiv where
  math := "The linear equiv between rightHandedWeyl and altRightHandedWeyl given
    by multiplying an element of rightHandedWeyl by the matrix  $\epsilon^{a_0 a_1} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ "
  deps := [``rightHandedWeyl, ``altRightHandedWeyl]
```

informal_lemma rightHandedWeylAltEquiv_equivariant[\[source\]](#)

```
informal_lemma rightHandedWeylAltEquiv_equivariant where
  math := "The linear equiv rightHandedWeylAltEquiv is equivariant with respect to the
    action of SL(2,C) on rightHandedWeyl and altRightHandedWeyl."
  deps := [``rightHandedWeylAltEquiv]
```

informal_definition leftAltWeylContraction[\[source\]](#)

```
informal_definition leftAltWeylContraction where
  math := "The linear map from leftHandedWeyl  $\otimes$  altLeftHandedWeyl to  $\mathbb{C}$  given by
    summing over components of leftHandedWeyl and altLeftHandedWeyl in the
    standard basis (i.e. the dot product)."
  physics := "The contraction of a left-handed Weyl fermion with a right-handed Weyl fermion.
    In index notation this is  $\psi_a \phi^a$ ."
  deps := [``leftHandedWeyl, ``altLeftHandedWeyl]
```

informal_lemma leftAltWeylContraction_invariant[\[source\]](#)

```
informal_lemma leftAltWeylContraction_invariant where
  math := "The contraction leftAltWeylContraction is invariant with respect to
    the action of SL(2,C) on leftHandedWeyl and altLeftHandedWeyl."
  deps := [``leftAltWeylContraction]
```

informal_definition altLeftWeylContraction[\[source\]](#)

```
informal_definition altLeftWeylContraction where
  math := "The linear map from altLeftHandedWeyl  $\otimes$  leftHandedWeyl to  $\mathbb{C}$  given by
    summing over components of altLeftHandedWeyl and leftHandedWeyl in the
    standard basis (i.e. the dot product)."
  physics := "The contraction of a left-handed Weyl fermion with a right-handed Weyl fermion.
    In index notation this is  $\phi^a \psi_a$ ."
  deps := [``leftHandedWeyl, ``altLeftHandedWeyl]
```

informal_lemma leftAltWeylContraction_symm_altLeftWeylContraction[\[source\]](#)

```
informal_lemma leftAltWeylContraction_symm_altLeftWeylContraction where
  math := "The linear map altLeftWeylContraction is leftAltWeylContraction composed
    with the braiding of the tensor product."
  deps := [``leftAltWeylContraction, ``altLeftWeylContraction]
```

informal_lemma altLeftWeylContraction_invariant[\[source\]](#)

```
informal_lemma altLeftWeylContraction_invariant where
  math := "The contraction altLeftWeylContraction is invariant with respect to
    the action of SL(2,C) on leftHandedWeyl and altLeftHandedWeyl."
  deps := [``altLeftWeylContraction]
```

informal_definition rightAltWeylContraction[\[source\]](#)

```
informal_definition rightAltWeylContraction where
  math := "The linear map from rightHandedWeyl  $\otimes$  altRightHandedWeyl to  $\mathbb{C}$  given by
    summing over components of rightHandedWeyl and altRightHandedWeyl in the
    standard basis (i.e. the dot product)."
```

physics := "The contraction of a right-handed Weyl fermion with a left-handed Weyl fermion.
In index notation this is $\psi_{\dot{a}} \varphi^{\dot{a}}$."

```
deps := [``rightHandedWeyl, ``altRightHandedWeyl]
```

informal_lemma rightAltWeylContraction_invariant[\[source\]](#)

```
informal_lemma rightAltWeylContraction_invariant where
  math := "The contraction rightAltWeylContraction is invariant with respect to
    the action of  $SL(2, \mathbb{C})$  on rightHandedWeyl and altRightHandedWeyl."
```

```
deps := [``rightAltWeylContraction]
```

informal_definition altRightWeylContraction[\[source\]](#)

```
informal_definition altRightWeylContraction where
  math := "The linear map from altRightHandedWeyl  $\otimes$  rightHandedWeyl to  $\mathbb{C}$  given by
    summing over components of altRightHandedWeyl and rightHandedWeyl in the
    standard basis (i.e. the dot product)."
```

physics := "The contraction of a right-handed Weyl fermion with a left-handed Weyl fermion.
In index notation this is $\varphi^{\dot{a}} \psi_{\dot{a}}$."

```
deps := [``rightHandedWeyl, ``altRightHandedWeyl]
```

informal_lemma rightAltWeylContraction_symm_altRightWeylContraction[\[source\]](#)

```
informal_lemma rightAltWeylContraction_symm_altRightWeylContraction where
  math := "The linear map altRightWeylContraction is rightAltWeylContraction composed
    with the braiding of the tensor product."
```

```
deps := [``rightAltWeylContraction, ``altRightWeylContraction]
```

informal_lemma altRightWeylContraction_invariant[\[source\]](#)

```
informal_lemma altRightWeylContraction_invariant where
  math := "The contraction altRightWeylContraction is invariant with respect to
    the action of  $SL(2, \mathbb{C})$  on rightHandedWeyl and altRightHandedWeyl."
```

```
deps := [``altRightWeylContraction]
```

2.14 HepLean.StandardModel.Basic

[HepLean/StandardModel/Basic.lean](#) on GitHub.

abbrev GaugeGroupI[\[source\]](#)

The global gauge group of the Standard Model with no discrete quotients. The I in the Name is an indication of the statement that this has no discrete quotients.

```
abbrev GaugeGroupI : Type
```


informal_definition gaugeGroupZ₆SubGroup[\[source\]](#)

```
informal_definition gaugeGroupZ6SubGroup where
  physics := "The subgroup of the un-quotiented gauge group which acts trivially on
    all particles in the standard model. "
  math := "The Z6-subgroup of ``GaugeGroupI with elements ( $\alpha^2 * I_3$ ,  $\alpha^{(-3)} * I_2$ ,  $\alpha$ ), where  $\alpha$ 
    is a sixth complex root of unity."
  ref := "https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI]
```

informal_definition GaugeGroupZ₆[\[source\]](#)

```
informal_definition GaugeGroupZ6 where
  physics := "The smallest possible gauge group of the Standard Model."
  math := "The quotient of ``GaugeGroupI by the Z6-subgroup `gaugeGroupZ6SubGroup`."
  ref := "https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI, ``StandardModel.gaugeGroupZ6SubGroup]
```

informal_definition gaugeGroupZ₂SubGroup[\[source\]](#)

```
informal_definition gaugeGroupZ2SubGroup where
  physics := "The Z2subgroup of the un-quotiented gauge group which acts trivially on
    all particles in the standard model. "
  math := "The Z2-subgroup of ``GaugeGroupI derived from the Z2 subgroup of `gaugeGroup
    Z6SubGroup`."
  ref := "https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI, ``StandardModel.gaugeGroupZ6SubGroup]
```

informal_definition GaugeGroupZ₂[\[source\]](#)

```
informal_definition GaugeGroupZ2 where
  physics := "The gauge group of the Standard Model with a Z2 quotient."
  math := "The quotient of ``GaugeGroupI by the Z2-subgroup `gaugeGroupZ2SubGroup`."
  ref := "https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI, ``StandardModel.gaugeGroupZ2SubGroup]
```

informal_definition gaugeGroupZ₃SubGroup[\[source\]](#)

```
informal_definition gaugeGroupZ3SubGroup where
  physics := "The Z3-subgroup of the un-quotiented gauge group which acts trivially on
    all particles in the standard model. "
  math := "The Z3-subgroup of ``GaugeGroupI derived from the Z3 subgroup of `gaugeGroup
    Z6SubGroup`."
  ref := "https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI, ``StandardModel.gaugeGroupZ6SubGroup]
```

informal_definition GaugeGroupZ₃[\[source\]](#)

```
informal_definition GaugeGroupZ3 where
  physics := "The gauge group of the Standard Model with a Z3-quotient."
  math := "The quotient of ``GaugeGroupI by the Z3-subgroup `gaugeGroupZ3SubGroup`."
  ref := "https://math.ucr.edu/home/baez/guts.pdf"
  deps := [``GaugeGroupI, ``StandardModel.gaugeGroupZ3SubGroup]
```

inductive GaugeGroupQuot[\[source\]](#)

Specifies the allowed quotients of $SU(3) \times SU(2) \times U(1)$ which give a valid gauge group of the Standard Model.

```
inductive GaugeGroupQuot : Type
| Z6 : GaugeGroupQuot
| Z2 : GaugeGroupQuot
| Z3 : GaugeGroupQuot
| I : GaugeGroupQuot
```

informal_lemma gaugeGroupI_lie[\[source\]](#)

```
informal_lemma gaugeGroupI_lie where
  math := "The gauge group `GaugeGroupI` is a Lie group.."
  deps := [``GaugeGroupI]
```

informal_lemma gaugeGroup_lie[\[source\]](#)

```
informal_lemma gaugeGroup_lie where
  math := "For every q in ``GaugeGroupQuot the group ``GaugeGroup q is a Lie group."
  deps := [``GaugeGroup]
```

informal_definition gaugeBundleI[\[source\]](#)

```
informal_definition gaugeBundleI where
  math := "The trivial principal bundle over SpaceTime with structure group ``GaugeGroupI."
  deps := [``GaugeGroupI, ``SpaceTime]
```

informal_definition gaugeTransformI[\[source\]](#)

```
informal_definition gaugeTransformI where
  math := "A global section of ``gaugeBundleI."
  deps := [``gaugeBundleI]
```

2.15 HepLean.StandardModel.HiggsBoson.Basic

[HepLean/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

def HiggsVec.ofReal[\[source\]](#)

Generating a Higgs vector from a real number, such that the norm-squared of that Higgs vector is the given real number.

```
def ofReal (a : ℝ) : HiggsVec
```

lemma ofReal_normSq[\[source\]](#)

```
@[simp]
lemma ofReal_normSq {a : ℝ} (ha : 0 ≤ a) : ||ofReal a|| ^ 2 = a
```

lemma toField_apply[\[source\]](#)

```
@[simp]
lemma HiggsVec.toField_apply (φ : HiggsVec) (x : SpaceTime) : (φ.toField x) = φ
```

def HiggsField.ofReal[\[source\]](#)

Generating a constant Higgs field from a real number, such that the norm-squared of that Higgs vector is the given real number.

```
def ofReal (a : ℝ) : HiggsField
```

def zero[\[source\]](#)

The higgs field which is all zero.

```
def zero : HiggsField
```

informal_lemma zero_is_zero_section[\[source\]](#)

```
informal_lemma zero_is_zero_section where
  physics := "The zero Higgs field is the zero section of the Higgs bundle."
  math := "The HiggsField `zero` defined by `ofReal 0`
    is the constant zero-section of the bundle `HiggsBundle`."
  deps := [StandardModel.HiggsField.zero]
```

2.16 HepLean.StandardModel.HiggsBoson.GaugeAction

[HepLean/StandardModel/HiggsBoson/GaugeAction.lean](#) on GitHub.

theorem rotatefst_real_snd_zero[\[source\]](#)

```
theorem rotatefst_real_snd_zero (φ : HiggsVec) :
  ∃ (g : GaugeGroupI), rep g φ = ![Complex.ofReal ||φ||, 0]
```

informal_lemma HiggsVec.gauge_orbit[\[source\]](#)

```
informal_lemma gauge_orbit where
  math := "There exists a `g` in ``GaugeGroupI such that `rep g φ = φ'` if and only if
    ||φ|| = ||φ'|."
  deps := [rotatefst_zero_snd_real]
```

informal_lemma stability_group_single[\[source\]](#)

```
informal_lemma stability_group_single where
  physics := "The Higgs boson breaks electroweak symmetry down to the electromagnetic force."
  math := "The stability group of the action of `rep` on `![0, Complex.ofReal ||φ||]`,
    for non-zero `||φ||` is the `SU(3) x U(1)` subgroup of
    `gaugeGroup := SU(3) x SU(2) x U(1)` with the embedding given by
    `(g, e^{i θ}) ↦ (g, diag (e ^ {3 * i θ}, e ^ {- 3 * i θ}), e^{i θ})`."
  deps := [StandardModel.HiggsVec, StandardModel.HiggsVec.rep]
```

informal_lemma stability_group[\[source\]](#)

```
informal_lemma stability_group where
  math := "The subgroup of `gaugeGroup := SU(3) x SU(2) x U(1)` which preserves every `HiggsVec`
    by the action of ``StandardModel.HiggsVec.rep is given by `SU(3) x Z_6` where Z_6
    is the subgroup of `SU(2) x U(1)` with elements `(α^(-3) * I_2, α)` where
    α is a sixth root of unity."
  deps := [``HiggsVec, ``rep]
```

informal_definition gaugeAction[\[source\]](#)

```
informal_definition gaugeAction where
  math := "The action of ``gaugeTransformI on ``HiggsField acting pointwise through
    ``HiggsVec.rep."
  deps := [``HiggsVec.rep, ``gaugeTransformI]
```

informal_lemma HiggsField.guage_orbit[\[source\]](#)

```
informal_lemma guage_orbit where
  math := "There exists a `g` in ``gaugeTransformI such that `gaugeAction g φ = φ` if and only
    if
    φ(x)^(† φ(x)) = φ'(x)^(† φ'(x))."
  deps := [``gaugeAction]
```

informal_lemma gauge_orbit_surject[\[source\]](#)

```
informal_lemma gauge_orbit_surject where
  math := "For every smooth map f from ``SpaceTime to ℝ such that `f` is positive semidefinite,
    there exists a Higgs field φ such that `f = φ^(† φ)`."
  deps := [``HiggsField, ``SpaceTime]
```

2.17 HepLean.StandardModel.HiggsBoson.PointwiseInnerProd

[HepLean/StandardModel/HiggsBoson/PointwiseInnerProd.lean](#) on GitHub.

lemma innerProd_neg_left[\[source\]](#)

```
@[simp]
lemma innerProd_neg_left (φ1 φ2 : HiggsField) : ⟨⟨- φ1, φ2⟩⟩_H = -⟨⟨φ1, φ2⟩⟩_H
```

lemma innerProd_neg_right[\[source\]](#)

```
@[simp]
lemma innerProd_neg_right (φ1 φ2 : HiggsField) : ⟨⟨φ1, - φ2⟩⟩_H = -⟨⟨φ1, φ2⟩⟩_H
```

lemma normSq_apply_im_zero[\[source\]](#)

```
@[simp]
lemma normSq_apply_im_zero (φ : HiggsField) (x : SpaceTime) :
  ((Complex.ofReal' ||φ x||) ^ 2).im = 0
```

lemma normSq_apply_re_self[\[source\]](#)

```
@[simp]
lemma normSq_apply_re_self (φ : HiggsField) (x : SpaceTime) :
  ((Complex.ofReal' ||φ x||) ^ 2).re = φ.normSq x
```

lemma ofReal_normSq[\[source\]](#)

```
lemma ofReal_normSq {a : ℝ} (ha : 0 ≤ a) (x : SpaceTime) : (ofReal a).normSq x = a
```

2.18 HepLean.StandardModel.HiggsBoson.Potential

[HepLean/StandardModel/HiggsBoson/Potential.lean](#) on GitHub.

structure Potential[\[source\]](#)

The parameters of the Higgs potential.

```
structure Potential where
```

```
  μ2 : ℝ
  ↑ : ℝ
```

def toFun[\[source\]](#)

The function corresponding to the Higgs potential.

```
def toFun (φ : HiggsField) (x : SpaceTime) : ℝ
```

lemma toFun_smooth[\[source\]](#)

The potential is smooth.

```
lemma toFun_smooth (φ : HiggsField) :
  Smooth  $\mathcal{I}(\mathbb{R}, \text{SpaceTime})$   $\mathcal{I}(\mathbb{R}, \mathbb{R})$  (fun x => P.toFun φ x)
```

def neg[\[source\]](#)

The Higgs potential formed by negating the mass squared and the quartic coupling.

```
def neg : Potential where
```

lemma toFun_neg[\[source\]](#)

```
@[simp]
lemma toFun_neg (φ : HiggsField) (x : SpaceTime) : P.neg.toFun φ x = - P.toFun φ x
```

lemma μ2_neg[\[source\]](#)

```
@[simp]
lemma μ2_neg : P.neg.μ2 = - P.μ2
```

lemma $\uparrow_{\text{neg}}$ [\[source\]](#)

@[simp]

lemma $\uparrow_{\text{neg}}$: $P.\text{neg}.\uparrow = - P.\uparrow$ **lemma toFun_zero**[\[source\]](#)

@[simp]

lemma toFun_zero (x : SpaceTime) : $P.\text{toFun HiggsField.zero } x = 0$ **lemma complete_square**[\[source\]](#)

lemma complete_square (h : $P.\uparrow \neq 0$) (ϕ : HiggsField) (x : SpaceTime) :
 $P.\text{toFun } \phi \ x = P.\uparrow * (\|\phi\|_H^2 x - P.\mu^2 / (2 * P.\uparrow))^2 - P.\mu^2^2 / (4 * P.\uparrow)$

lemma as_quad[\[source\]](#)

lemma as_quad (ϕ : HiggsField) (x : SpaceTime) :
 $P.\uparrow * \|\phi\|_H^2 x * \|\phi\|_H^2 x + (- P.\mu^2) * \|\phi\|_H^2 x + (- P.\text{toFun } \phi \ x) = 0$

lemma toFun_eq_zero_iff[\[source\]](#)

lemma toFun_eq_zero_iff (h : $P.\uparrow \neq 0$) (ϕ : HiggsField) (x : SpaceTime) :
 $P.\text{toFun } \phi \ x = 0 \leftrightarrow \phi \ x = 0 \vee \|\phi\|_H^2 x = P.\mu^2 / P.\uparrow$

def quadDiscrim[\[source\]](#)

The discriminant of the quadratic equation formed by the Higgs potential.

def quadDiscrim (ϕ : HiggsField) (x : SpaceTime) : $\mathbb{R}$

lemma quadDiscrim_nonneg[\[source\]](#)

The discriminant of the quadratic formed by the potential is non-negative.

lemma quadDiscrim_nonneg (h : $P.\uparrow \neq 0$) (ϕ : HiggsField) (x : SpaceTime) :
 $0 \leq P.\text{quadDiscrim } \phi \ x$

lemma quadDiscrim_eq_sqrt_mul_sqrt[\[source\]](#)

lemma quadDiscrim_eq_sqrt_mul_sqrt (h : $P.\uparrow \neq 0$) (ϕ : HiggsField) (x : SpaceTime) :
 $P.\text{quadDiscrim } \phi \ x = \text{Real.sqrt } (P.\text{quadDiscrim } \phi \ x) * \text{Real.sqrt } (P.\text{quadDiscrim } \phi \ x)$

lemma quadDiscrim_eq_zero_iff[\[source\]](#)

lemma quadDiscrim_eq_zero_iff (h : $P.\uparrow \neq 0$) (ϕ : HiggsField) (x : SpaceTime) :
 $P.\text{quadDiscrim } \phi \ x = 0 \leftrightarrow P.\text{toFun } \phi \ x = - P.\mu^2^2 / (4 * P.\uparrow)$

lemma quadDiscrim_eq_zero_iff_normSq[\[source\]](#)

lemma quadDiscrim_eq_zero_iff_normSq (h : $P.\uparrow \neq 0$) (ϕ : HiggsField) (x : SpaceTime) :
 $P.\text{quadDiscrim } \phi \ x = 0 \leftrightarrow \|\phi\|_H^2 x = P.\mu^2 / (2 * P.\uparrow)$

lemma neg_↓_quadDiscrim_zero_bound[\[source\]](#)

```
lemma neg_↓_quadDiscrim_zero_bound (h : P.↓ < 0) (φ : HiggsField) (x : SpaceTime) :
  P.toFun φ x ≤ - P.μ2 ^ 2 / (4 * P.↓)
```

lemma pos_↓_quadDiscrim_zero_bound[\[source\]](#)

```
lemma pos_↓_quadDiscrim_zero_bound (h : 0 < P.↓) (φ : HiggsField) (x : SpaceTime) :
  - P.μ2 ^ 2 / (4 * P.↓) ≤ P.toFun φ x
```

lemma neg_↓_toFun_neg[\[source\]](#)

```
lemma neg_↓_toFun_neg (h : P.↓ < 0) (φ : HiggsField) (x : SpaceTime) :
  (0 < P.μ2 ∧ P.toFun φ x ≤ 0) ∨ P.μ2 ≤ 0
```

lemma pos_↓_toFun_pos[\[source\]](#)

```
lemma pos_↓_toFun_pos (h : 0 < P.↓) (φ : HiggsField) (x : SpaceTime) :
  (P.μ2 < 0 ∧ 0 ≤ P.toFun φ x) ∨ 0 ≤ P.μ2
```

lemma neg_↓_sol_exists_iff[\[source\]](#)

```
lemma neg_↓_sol_exists_iff (h↓ : P.↓ < 0) (c : ℝ) : (∃ φ x, P.toFun φ x = c) ↔ (0 < P.μ2 ∧ c ≤
  0) ∨
  (P.μ2 ≤ 0 ∧ c ≤ - P.μ2 ^ 2 / (4 * P.↓))
```

lemma pos_↓_sol_exists_iff[\[source\]](#)

```
lemma pos_↓_sol_exists_iff (h↓ : 0 < P.↓) (c : ℝ) : (∃ φ x, P.toFun φ x = c) ↔ (P.μ2 < 0 ∧ 0 ≤
  c) ∨
  (0 ≤ P.μ2 ∧ - P.μ2 ^ 2 / (4 * P.↓) ≤ c)
```

def IsBounded[\[source\]](#)

The proposition on the coefficients for a potential to be bounded.

```
def IsBounded : Prop
```

lemma isBounded_↓_nonneg[\[source\]](#)

```
lemma isBounded_↓_nonneg (h : P.IsBounded) :
  0 ≤ P.↓
```

lemma isBounded_of_↓_pos[\[source\]](#)

```
lemma isBounded_of_↓_pos (h : 0 < P.↓) : P.IsBounded
```

informal_lemma isBounded_iff_of_↓_zero[\[source\]](#)

```
informal_lemma isBounded_iff_of_↓_zero where
  physics := "When there is no quartic coupling, the potential is bounded iff the mass squared
  is"
```

```

    non-positive."
math := "For `P : Potential` then P.IsBounded if and only if  $P.\mu_2 \leq 0$ .
    That is to say ` $- P.\mu_2 * \|\phi\|_H^2 x$ ` is bounded below if and only if ` $P.\mu_2 \leq 0$ `."
deps := [ `StandardModel.HiggsField.Potential.IsBounded, `StandardModel.HiggsField.Potential]

```

lemma eq_zero_iff_of_μSq_nonpos_↑_pos

[\[source\]](#)

```

lemma eq_zero_iff_of_μSq_nonpos_↑_pos (h↑ : 0 < P.↑) (hμ2 : P.μ2 ≤ 0) (φ : HiggsField)
  (x : SpaceTime) : P.toFun φ x = 0 ↔ φ x = 0

```

lemma isMin0n_iff_of_μSq_nonpos_↑_pos

[\[source\]](#)

```

lemma isMin0n_iff_of_μSq_nonpos_↑_pos (h↑ : 0 < P.↑) (hμ2 : P.μ2 ≤ 0) (φ : HiggsField)
  (x : SpaceTime) : IsMin0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x)
  ↔ P.toFun φ x = 0

```

lemma isMin0n_iff_field_of_μSq_nonpos_↑_pos

[\[source\]](#)

```

lemma isMin0n_iff_field_of_μSq_nonpos_↑_pos (h↑ : 0 < P.↑) (hμ2 : P.μ2 ≤ 0) (φ : HiggsField)
  (x : SpaceTime) : IsMin0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x)
  ↔ φ x = 0

```

lemma isMin0n_iff_of_μSq_nonneg_↑_pos

[\[source\]](#)

```

lemma isMin0n_iff_of_μSq_nonneg_↑_pos (h↑ : 0 < P.↑) (hμ2 : 0 ≤ P.μ2) (φ : HiggsField)
  (x : SpaceTime) : IsMin0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x) ↔
  P.toFun φ x = - P.μ2 ^ 2 / (4 * P.↑)

```

lemma isMin0n_iff_field_of_μSq_nonneg_↑_pos

[\[source\]](#)

```

lemma isMin0n_iff_field_of_μSq_nonneg_↑_pos (h↑ : 0 < P.↑) (hμ2 : 0 ≤ P.μ2) (φ : HiggsField)
  (x : SpaceTime) : IsMin0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x) ↔
   $\|\phi\|_H^2 x = P.\mu_2 / (2 * P.\uparrow)$ 

```

theorem isMin0n_iff_field_of_↑_pos

[\[source\]](#)

```

theorem isMin0n_iff_field_of_↑_pos (h↑ : 0 < P.↑) (φ : HiggsField) (x : SpaceTime) :
  IsMin0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x) ↔
  (0 ≤ P.μ2 ∧  $\|\phi\|_H^2 x = P.\mu_2 / (2 * P.\uparrow)$ ) ∨ (P.μ2 < 0 ∧ φ x = 0)

```

lemma isMax0n_iff_isMin0n_neg

[\[source\]](#)

```

lemma isMax0n_iff_isMin0n_neg (φ : HiggsField) (x : SpaceTime) :
  IsMax0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x) ↔
  IsMin0n (fun (φ, x) => P.neg.toFun φ x) Set.univ (φ, x)

```

lemma isMax0n_iff_field_of_↑_neg

[\[source\]](#)

```

lemma isMax0n_iff_field_of_↑_neg (h↑ : P.↑ < 0) (φ : HiggsField) (x : SpaceTime) :
  IsMax0n (fun (φ, x) => P.toFun φ x) Set.univ (φ, x) ↔
  (P.μ2 ≤ 0 ∧  $\|\phi\|_H^2 x = P.\mu_2 / (2 * P.\uparrow)$ ) ∨ (0 < P.μ2 ∧ φ x = 0)

```


2.19 `scripts.MetaPrograms.check_rfl`

[scripts/MetaPrograms/check_rfl.lean](#) on GitHub.

def name? [\[source\]](#)

```
def name? (t : TacticInfo) : Option Name
```

def isSubstantive [\[source\]](#)

Decide whether a tactic is "substantive", or is merely a tactic combinator (e.g. `by`, `;`, multiline tactics, parenthesized tactics).

```
def isSubstantive (t : TacticInfo) : Bool
```

def visitTacticInfo [\[source\]](#)

```
def visitTacticInfo (file : FilePath) (ci : ContextInfo) (ti : TacticInfo) : MetaM Unit
```

def processAllFiles [\[source\]](#)

```
unsafe def processAllFiles : IO Unit
```

def main [\[source\]](#)

```
unsafe def main (args : List String) : IO Unit
```

2.20 `scripts.MetaPrograms.free_simps`

[scripts/MetaPrograms/free_simps.lean](#) on GitHub.

def name? [\[source\]](#)

```
def name? (t : TacticInfo) : Option Name
```

def isSimp [\[source\]](#)

Decide whether a tactic is "substantive", or is merely a tactic combinator (e.g. `by`, `;`, multiline tactics, parenthesized tactics).

```
def isSimp (t : TacticInfo) : Bool
```

def visitTacticInfo [\[source\]](#)

```
def visitTacticInfo (file : FilePath) (ci : ContextInfo) (ti : TacticInfo) : MetaM Unit
```

def processAllFiles [\[source\]](#)

```
unsafe def processAllFiles : IO Unit
```

def main [\[source\]](#)

```
unsafe def main (args : List String) : IO Unit
```

2.21 scripts.MetaPrograms.informal

[scripts/MetaPrograms/informal.lean](#) on GitHub.

def getConst [\[source\]](#)

```
def getConst (imp : Import) : IO (Array (Import × ConstantInfo))
```

def getLineNumber [\[source\]](#)

```
def getLineNumber (c : Name) : MetaM Nat
```

def getModule [\[source\]](#)

```
def getModule (c : Name) : MetaM Name
```

def getConstInfo [\[source\]](#)

```
def getConstInfo (n : Name) : MetaM ConstantInfo
```

def getDocString [\[source\]](#)

Gets the docstring from a name, if it exists, otherwise the string "No docstring."

```
def getDocString (n : Name) : MetaM String
```

def depToString [\[source\]](#)

```
def depToString (d : Name) : MetaM String
```

def depToWebString [\[source\]](#)

```
def depToWebString (d : Name) : MetaM String
```

def constantInfoToInformalLemma [\[source\]](#)

Takes a ConstantInfo corresponding to a InformalLemma and returns the corresponding InformalLemma.

```
unsafe def constantInfoToInformalLemma (c : ConstantInfo) : MetaM InformalLemma
```

def constantInfoToInformalDefinition [\[source\]](#)

Takes a ConstantInfo corresponding to a InformalDefinition and returns the corresponding InformalDefinition.

```
unsafe def constantInfoToInformalDefinition (c : ConstantInfo) : MetaM InformalDefinition
```

def informalDependencies [\[source\]](#)

```
unsafe def informalDependencies (c : ConstantInfo) : MetaM (Array Name)
```

def informalLemmaToString [\[source\]](#)

```
unsafe def informalLemmaToString (c : Import × ConstantInfo) : MetaM String
```

def informalLemmaToWebString [\[source\]](#)

```
unsafe def informalLemmaToWebString (c : Import × ConstantInfo) : MetaM String
```

def informalDefToString [\[source\]](#)

```
unsafe def informalDefToString (c : Import × ConstantInfo) : MetaM String
```

def informalDefToWebString [\[source\]](#)

```
unsafe def informalDefToWebString (c : Import × ConstantInfo) : MetaM String
```

def informalToString [\[source\]](#)

```
unsafe def informalToString (c : Import × ConstantInfo) : MetaM String
```

def informalToWebString [\[source\]](#)

```
unsafe def informalToWebString (c : Import × ConstantInfo) : MetaM String
```

def informalFileHeader [\[source\]](#)

```
def informalFileHeader : String
```

def importToInformal [\[source\]](#)

Takes an import and outputs the list of ConstantInfo corresponding to an informal definition or lemma in that import, sorted by line number.

```
def importToInformal (i : Import) : MetaM (Array (Import × ConstantInfo))
```

def importToString [\[source\]](#)

```
unsafe def importToString (i : Import) : MetaM String
```

def importToWebString [\[source\]](#)

```
unsafe def importToWebString (i : Import) : MetaM String
```

def formalToNode[\[source\]](#)*Turns a formal definition or lemma into a node of a dot graph.*

```
def formalToNode (nameSpaces : Array Name) (d : Name) : MetaM String
```

def informalLemmaToNode[\[source\]](#)

```
unsafe def informalLemmaToNode (nameSpaces : Array Name) (c : Import × ConstantInfo) : MetaM String
```

def informalDefToNode[\[source\]](#)

```
unsafe def informalDefToNode (nameSpaces : Array Name) (c : Import × ConstantInfo) : MetaM String
```

def informalToNode[\[source\]](#)

```
unsafe def informalToNode (nameSpaces : Array Name) (c : Import × ConstantInfo) : MetaM String
```

def informalLemmaToEdges[\[source\]](#)

```
unsafe def informalLemmaToEdges (c : Import × ConstantInfo) : MetaM (String)
```

def informalDefToEdges[\[source\]](#)

```
unsafe def informalDefToEdges (c : Import × ConstantInfo) : MetaM (String)
```

def informalToEdges[\[source\]](#)

```
unsafe def informalToEdges (c : Import × ConstantInfo) : MetaM (String)
```

def namespaceToCluster[\[source\]](#)

```
unsafe def namespaceToCluster (name : Name) : MetaM String
```

def mkDot[\[source\]](#)

```
unsafe def mkDot (imports : Array Import) : MetaM String
```

def formalToHTML[\[source\]](#)

```
def formalToHTML (d : Name) : MetaM String
```

def informalLemmaToHTML[\[source\]](#)

```
unsafe def informalLemmaToHTML (c : Import × ConstantInfo) : MetaM String
```

def informalDefToHTML[\[source\]](#)

```
unsafe def informalDefToHTML (c : Import × ConstantInfo) : MetaM String
```

def informalToHTML[\[source\]](#)

```
unsafe def informalToHTML (c : Import × ConstantInfo) : MetaM String
```

def toHTML[\[source\]](#)

```
unsafe def toHTML (imports : Array Import) : MetaM String
```

def main[\[source\]](#)

```
unsafe def main (args : List String) : IO UInt32
```